Na poprzednich zajęciach:

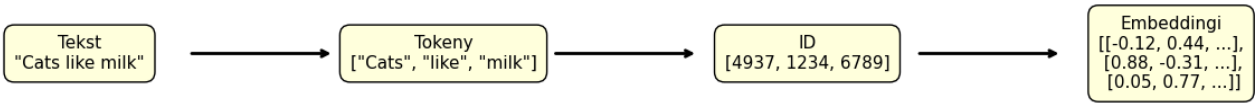
- **tokenizacja** – podział tekstu na słowa / tokeny,
- **spaCy Matcher** – znajdowanie wzorców na podstawie części mowy i cech morfologicznych.

Wektory słów

Tokenizacja a wektory słów

Tokenizacja -> Podziel tekst na kawałki (ID ze słownika)

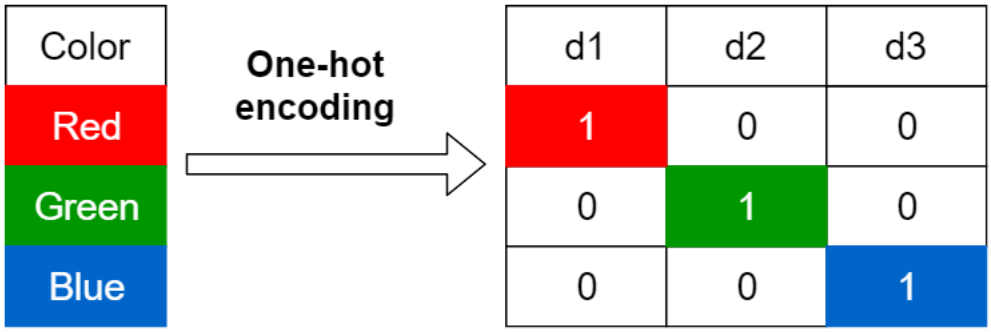
Embedding -> Przedstaw każde słowo w postaci liczb, aby algorytm mógł z nim dalej pracować



Komputer nie „rozumie" tekstu, tylko operuje na liczbach. Zatem każde słowo (np. „król", „kot", „miłość") trzeba jakoś zamienić na liczby, żeby można było nim manipulować w modelu.

Najprostszy pomysł — **one-hot encoding**

```
słownik = [kot, król, królowa]
"król" → [0, 1, 0]
```



Dlaczego one-hot encoding jest problematyczny?

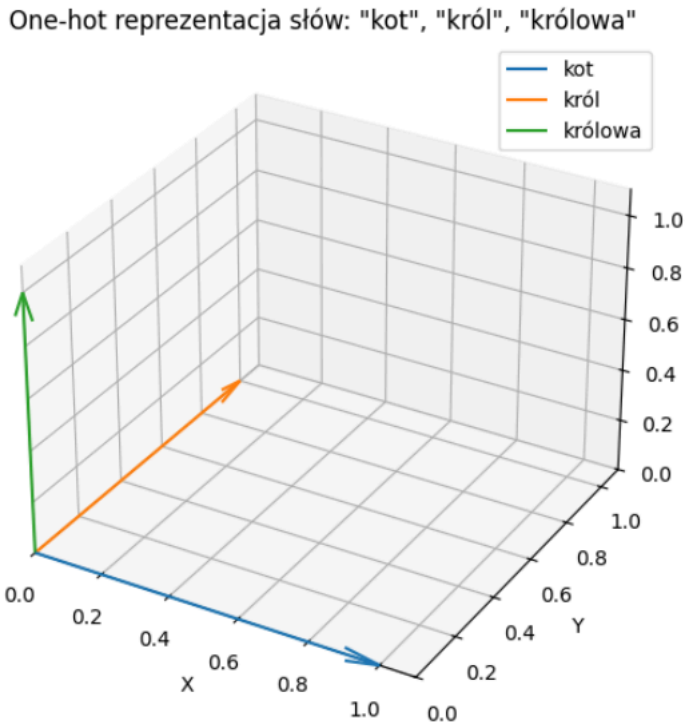
❌ Bardzo duży wymiar - Jeśli mamy słownik np. 100 000 słów, każdy wektor ma długość 100 000 — to ogromne marnotrawstwo pamięci.

❌ Brak informacji o znaczeniu - Wszystkie wektory są od siebie tak samo odległe — np. "kot" i "pies" są tak samo różne jak "kot" i "samolot". Nie ma żadnego pojęcia o podobieństwie semantycznym (znaczeniowym).

❌ Brak możliwości uogólniania - Model nie może „zrozumieć", że pewne słowa mają podobne znaczenie — każde słowo jest całkowicie niezależne.

```
słownik = [kot, król, królowa]
"kot" → [1, 0, 0]
```

```
"król" → [0, 1, 0]
"królowa" → [0, 0, 1]
```



Widzimy, że każdy wektor idzie w swoją stronę, każdy jest tak samo odległy od siebie (90 stopni). Żadne dwa nie są do siebie bliżej ani dalej — odległość między każdym z nich jest taka sama.

W naszym przykładzie: kot, król i królowa są tak samo podobni i różni = nie ma wartości semantycznej.

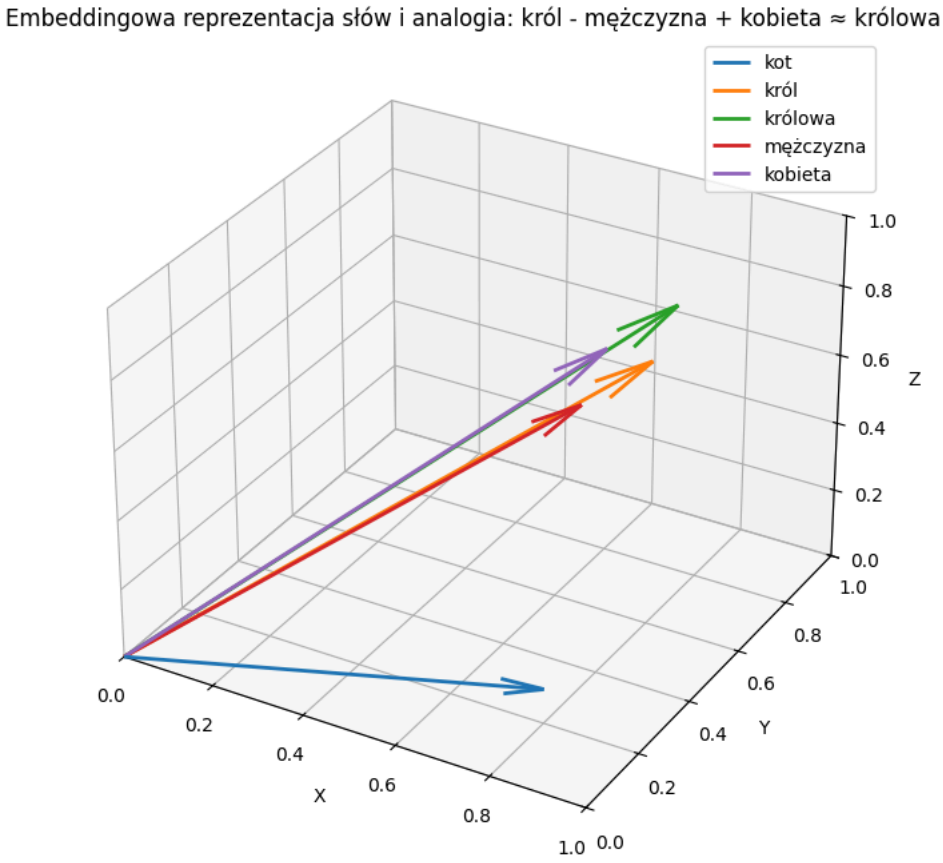
Embedingi

Embeddingi rozwiązują ten problem. W embeddingach uczymy model, żeby słowa, które występują w podobnych kontekstach, miały **podobne wektory**.

Dwa wektory są podobne, kiedy wskazują w podobnym kierunku w przestrzeni wielowymiarowej. Czyli: jeśli narysujesz te wektory jako strzałki zaczynające się w tym samym punkcie (np. w punkcie (0,0,0)) to mają **mały kąt między sobą**.

Przykład:

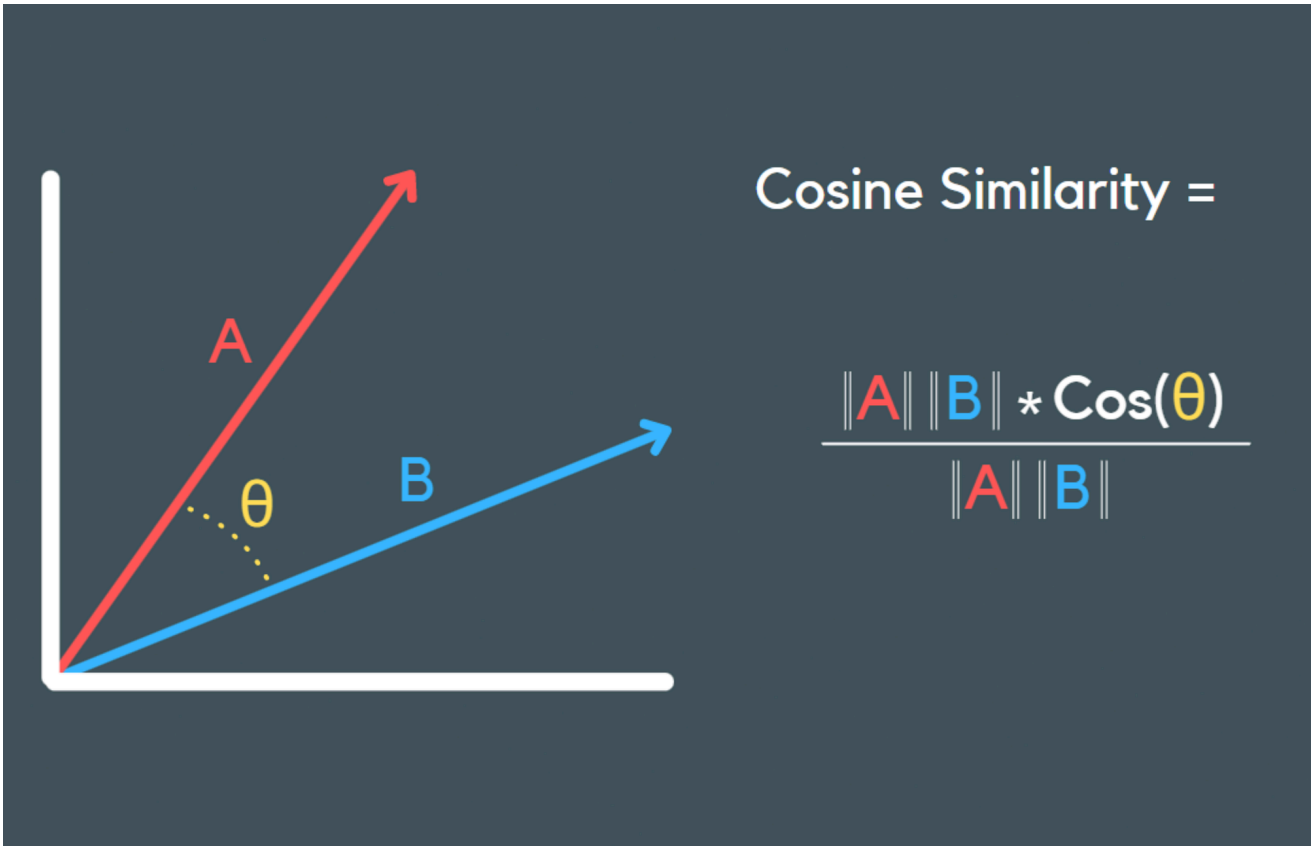
```
"king" → [0.25, 0.88, -0.12, 0.40, ...]
"queen" → [0.22, 0.90, -0.10, 0.45, ...]
"cat" → [-0.70, 0.11, 0.45, 0.20, ...]
```




```
król – mężczyzna + kobieta ≈ królowa
```

Teraz punkty w przestrzeni układają się tak, że słowa o podobnym znaczeniu są blisko siebie. To jest właśnie **semantyczne podobieństwo** — podobieństwo w sensie znaczenia, a nie liter czy indeksów w tabeli.

Cosinusowe podobieństwo — jak mierzymy „bliskość znaczeń”



Cosinusowe podobieństwo to sposób na sprawdzenie, czy dwa wektory są „w tym samym kierunku":

- 1 → bardzo podobne (np. „kot" i „pies")
- 0 → brak podobieństwa (np. „kot" i „okno")
- -1 → całkowite przeciwieństwo

Ważne: Patrzymy nie na długość wektorów, tylko na kąt między nimi. Dwa wektory, które „patrzą" w tym samym kierunku, oznaczają słowa o podobnym znaczeniu.

Skąd się biorą te wektory?

Z treningu modelu językowego (np. Word2Vec, GloVe, FastText, BERT). Model uczy się na dużych korpusach tekstu, obserwując konteksty słów.

Efekt:

„king" i „queen" często występują w podobnych zdaniach (np. royal palace, throne, reign) → mają podobne wektory.

„dog" i „cat" też → inna grupa semantyczna.

Trochę praktyki

Zaczniemy od zapoznania się z biblioteką **gensim** Jej nazwa pochodzi od "GENerate SIMilar" — bo jej głównym celem jest znajdowanie podobieństwa między dokumentami lub słowami.

Użyjemy klasy **KeyedVectors** - to strukturą danych w gensim, która przechowuje wektory słów (embeddingi) wytrenowane dowolnym modelem. Czyli po prostu taki kontener do pracy z wektorami - niezależnie od tego, jak one zostały wytrenowane.

```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.13/dist-packages (from gensim) (2.1.3)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.13/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.13/dist-packages (from gensim) (8.0.2)
Requirement already satisfied: wrapt in /usr/local/lib/python3.13/dist-packages (from smart_open>=1.8.1->gensim) (2.5.0)
Downloading gensim-4.4.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.8/27.8 MB 43.2 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
import gensim
from gensim.models import KeyedVectors
```

weźmiemy sobie polskie pre-trenowane wektory: http://dsmodels.nlp.ipipan.waw.pl pobieramy plik:

```
!wget http://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-
```

```
--2026-10-15 16:40:42--  http://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz
Resolving dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)... 213.135.36.94
Connecting to dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)|213.135.36.94|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz [following]
--2026-10-15 16:40:42--  https://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz
Connecting to dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)|213.135.36.94|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2152612134 (2.0G) [application/octet-stream]
Saving to: ‘nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz’

nkjp+wiki-forms-res 100%[===================>]   2.00G  15.8MB/s    in 2m 35s

2026-10-15 16:43:19 (13.2 MB/s) - ‘nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz’ saved [2152612134/2152612134]
```

Rozpakowujemy dane

```
!gzip -d "nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz"
```

Badamy format pliku

```
!head -n 100 nkjp+wiki-forms-restricted-300-skipg-ns.txt
#Pierwsza linia - lość słów (1982479), rozmiar wektora(300)
#Każda kolejna linia - słowo, liczby reperezentujące wektor
```

```
1982479 300
jest -0.127787 0.008677 -0.404733 0.076550 -0.071382 -0.005960 -0.357351 -0.104665 0.344031 0.607296 0.304090 0.205259 0.2
to -0.083874 0.094876 0.122346 -0.006300 0.029354 0.023014 -0.037953 -0.212705 0.004550 0.139284 -0.162424 -0.021550 0.503
jak -0.007359 0.272532 -0.029307 0.115231 0.037336 0.089230 -0.321993 -0.019786 0.059345 0.134049 -0.138555 -0.172492 0.22
tym -0.076970 0.330795 0.037607 0.263601 0.237870 0.206891 -0.087761 -0.124613 0.001548 0.455860 0.182234 0.100705 0.31159
pl 0.299866 -0.339672 0.235895 0.287754 0.050424 -0.250907 -0.732772 0.260977 0.158818 -0.167742 -0.092325 0.085794 -0.226
roku -0.099483 0.174201 0.163370 0.213244 -0.607008 0.061205 -0.012465 0.130327 0.133978 0.364153 -0.284065 -0.366834 0.10
co -0.149627 -0.015899 0.246352 0.237757 -0.102143 0.157166 0.134786 -0.327036 0.054293 0.066131 -0.106071 0.033730 0.2840
są -0.128698 0.264488 -0.170631 0.159279 0.102607 0.168519 -0.610110 -0.100576 0.230376 0.611208 0.078295 0.046868 0.28176
ma -0.192878 0.188585 -0.091651 0.622142 0.043449 -0.210917 -0.412563 -0.045221 0.091658 0.312261 0.097047 0.052997 0.1177
1 -0.030362 0.346969 0.115749 0.270884 -0.085511 -0.178087 -0.028511 0.231794 0.223650 -0.110866 0.121705 -0.276777 0.5584
tego 0.052118 0.142630 -0.189004 0.197736 0.089805 0.167542 0.164978 -0.074676 0.001518 0.434493 -0.000536 0.054462 0.0815
tak 0.056147 0.150462 0.144555 0.099626 0.180400 -0.114361 -0.087904 0.075873 0.013473 0.229449 -0.023280 -0.218663 0.2787
był -0.138966 -0.027094 -0.093754 0.362354 -0.187679 -0.140251 -0.265825 -0.144974 0.068318 0.501494 0.009753 -0.083040 -0
było 0.124044 0.083316 0.147207 0.102246 -0.169104 0.205269 -0.104774 0.159169 0.061468 0.414562 -0.176555 -0.222450 0.488
który 0.045730 -0.223227 0.142384 0.228910 -0.267745 -0.152346 0.112955 -0.637149 0.003621 0.267546 -0.049973 0.006724 0.2
które -0.064818 0.049899 0.156545 0.227973 -0.045496 -0.072168 0.139552 -0.518176 -0.203038 0.104202 -0.350353 -0.219377 0
bardzo 0.019330 0.000516 0.038659 0.263874 -0.068502 -0.236071 0.035564 0.108277 0.112309 0.128411 0.130803 0.077926 0.420
tej -0.203367 0.156400 0.378099 0.093366 0.034211 0.005359 -0.360963 -0.244544 0.158451 0.407083 0.005968 -0.173815 0.2925
być -0.388348 -0.183055 -0.341474 0.105108 0.009847 0.076565 -0.111895 -0.159213 0.151446 0.330194 -0.167591 -0.033831 0.3
ten -0.116808 -0.097397 0.178635 0.161104 -0.101448 -0.286287 0.000851 -0.089643 -0.099683 0.311623 0.041602 0.096606 0.52
lat -0.132265 0.069441 -0.023357 0.272255 0.089522 0.087004 -0.036773 0.363295 -0.012658 0.334243 -0.136671 0.032483 0.131
może -0.194437 0.111466 -0.175709 0.174817 0.004653 -0.081439 0.023191 -0.161892 -0.022989 0.350862 -0.273439 -0.168024 0.
napisał 0.045800 -0.121663 0.087540 0.294228 -0.198758 -0.355542 -0.269791 -0.040715 -0.241816 0.764780 -0.070768 0.184902
r -0.288940 0.078715 0.265563 0.199849 -0.564114 0.038433 -0.022739 0.223896 0.246777 0.329983 -0.183803 -0.368775 -0.2083
news -0.105218 -0.316592 -0.010492 0.534373 -0.059806 -0.029232 -0.456143 0.259730 -0.012996 -0.004282 -0.097955 0.296070
była -0.558429 -0.162914 -0.389680 0.490204 -0.208617 -0.067719 -0.181986 0.015817 -0.032012 0.495107 -0.150884 -0.004947
align -0.816865 -0.915994 -0.561109 -0.524508 0.483104 -0.616137 -0.032534 -0.310422 -0.252538 -0.369867 -0.545488 0.11628
m 0.113424 0.406340 0.020684 0.626952 0.049988 -0.081138 0.239041 -0.031673 0.225328 0.272472 0.040784 0.106439 0.229499 -
został -0.409356 0.023851 0.054331 0.513416 -0.323213 -0.127434 -0.152621 -0.376789 0.041764 0.392734 -0.064568 -0.098157
mówi -0.126111 -0.109324 0.021549 -0.319608 0.047051 -0.262192 -0.248026 -0.020262 -0.263484 0.299982 0.073016 0.196141 0.
gdy -0.209188 -0.034081 0.019719 -0.197358 0.044856 -0.062860 0.079851 -0.309522 0.054711 0.134339 -0.191803 -0.185495 0.1
zł -0.105835 -0.582940 0.315753 -0.108083 -0.081552 -0.253764 -0.352474 -0.017699 0.332934 0.155621 0.327957 0.052505 -0.0
tych -0.109366 0.336909 -0.149185 0.091934 0.123739 -0.092410 0.050255 -0.103689 0.022349 0.562332 0.038495 -0.240496 0.09
te -0.240424 0.216611 0.280844 0.344817 0.051054 -0.136962 -0.151594 -0.112780 -0.200979 0.060350 -0.103896 -0.165631 0.25
2 -0.031078 0.100914 -0.015150 0.364988 -0.144344 -0.330129 -0.100422 0.157612 0.094501 0.039627 0.016108 -0.248943 0.4496
która -0.252817 -0.436101 0.093817 0.535854 -0.151353 -0.236457 0.052471 -0.357497 0.154315 0.140891 -0.184934 0.031189 0.
którzy -0.097952 0.044214 0.245157 0.086928 -0.100278 -0.046244 -0.129638 -0.458666 0.205760 0.271232 -0.302552 -0.092620
tu 0.175985 0.206204 0.332571 0.386022 0.152724 -0.204439 -0.022180 -0.404816 -0.215863 0.465108 0.070794 -0.281779 0.0680
pan 0.094128 -0.039227 -0.138648 0.228942 0.236455 0.010758 -0.288527 0.243305 -0.055104 0.201406 0.126915 0.205154 0.2184
gdzie 0.480528 0.206866 0.032789 0.228872 0.062058 -0.050831 0.260197 -0.502796 0.000242 0.467241 -0.139728 -0.139605 -0.0
3 -0.149336 0.098354 0.141462 0.238623 -0.106425 -0.405156 -0.118849 0.137495 0.068282 -0.014042 -0.035352 -0.219326 0.464
Http -0.038438 -0.755245 0.053644 0.096092 0.243646 -0.129186 0.043364 -0.131172 0.179391 -0.592744 -0.285257 0.523395 -0.
pracy -0.339222 0.225263 0.286258 0.264719 0.531696 0.053109 -0.272201 -0.205720 0.279595 0.215903 -0.055952 0.223827 0.28
były -0.102505 0.023722 -0.113885 0.347774 -0.238682 0.120603 -0.515847 -0.109189 0.041271 0.577869 -0.192093 -0.198294 0.
tam 0.321448 0.278772 0.074819 0.348238 0.167050 -0.159583 0.092537 -0.159371 -0.053519 0.518208 -0.075027 -0.171407 0.091
osób -0.577304 -0.309684 0.332576 -0.130018 0.225140 0.141721 0.106455 0.383576 0.163807 0.312394 -0.101676 -0.453438 -0.2
in -0.038397 0.109123 -0.062359 0.729193 0.147004 -0.184472 -0.049459 -0.168531 0.079803 0.369984 -0.042513 0.050099 0.225
```

```
kiedy -0.165688 0.137390 -0.016459 -0.225001 -0.075221 0.031155 0.131225 -0.234085 0.040119 0.127815 -0.288528 -0.170036 0
miał -0.192369 0.167752 0.068614 0.873184 -0.038002 -0.354534 -0.153245 -0.218563 -0.147391 0.394977 -0.348807 -0.341366 0
teraz -0.006893 0.157881 0.011489 -0.106358 -0.040639 -0.149671 0.104452 -0.086911 -0.192279 0.147113 -0.147597 -0.045625
użytkownik -0.117832 -0.403967 -0.140548 -0.193260 -0.230561 -0.049713 -0.490938 0.392071 -0.173403 0.324775 -0.338664 0.5
wiadomości -0.148818 0.034035 0.336113 0.281813 -0.417340 0.033253 -0.460107 0.398972 -0.034125 0.381669 -0.340300 0.11402
mają -0.081021 0.322764 -0.017296 0.622897 0.183037 -0.163510 -0.295295 -0.172757 0.283006 0.324483 -0.087236 0.111205 0.2
mam 0.112946 0.213810 0.179386 0.489940 0.108140 -0.057553 -0.173646 -0.022765 -0.159182 0.024074 0.031143 0.101109 -0.057
sie -0.238574 -0.122860 -0.116555 0.001156 -0.170150 0.075555 -0.074249 0.205734 -0.122618 -0.354006 -0.157999 -0.143338 -
których 0.084602 0.125160 0.002428 0.174309 0.022871 -0.108204 0.060451 -0.468605 -0.032858 0.468381 0.056205 -0.247217 0
```

(to może chwilę zająć)

```
#Ładowanie wektorów z pliku do naszego KeyedVectors
wv = KeyedVectors.load_word2vec_format('nkjp+wiki-forms-restricted-300-skipg-ns.txt')
```

```
wv.key_to_index.keys()
wv['dom']
```

```
array([-0.39294 , -0.156333,  0.017816, -0.208752,  0.442839, -0.389071,
       -0.22354 ,  0.119761,  0.201861,  0.136966,  0.005829, -0.036992,
       -0.075693, -0.424958,  0.031507, -0.214998, -0.11862 , -0.059799,
       -0.150971, -0.176567,  0.444136,  0.18925 ,  0.241265, -0.248733,
       -0.176206,  0.579075,  0.031083,  0.253453,  0.101754,  0.144021,
       -0.169662, -0.005122,  0.056225,  0.078038, -0.557848, -0.365922,
       -0.108863,  0.322074, -0.178232,  0.289982, -0.205699,  0.001991,
        0.001029, -0.074944,  0.173145,  0.044346, -0.032673, -0.060124,
        0.152098, -0.293121, -0.280878,  0.377744,  0.402627, -0.737609,
        0.020663, -0.01301 ,  0.134813, -0.052369,  0.046645, -0.024955,
        0.100392, -0.295437, -0.418354,  0.396864,  0.232666,  0.022819,
       -0.018474,  0.385309, -0.084371,  0.269105,  0.149824, -0.161652,
        0.282115,  0.177849, -0.241524, -0.240634, -0.111456,  0.070668,
       -0.216499, -0.139156, -0.576622, -0.136423,  0.403575,  0.155968,
        0.218948, -0.002413,  0.123077,  0.102393,  0.741444,  0.390994,
       -0.110964,  0.134329,  0.265171,  0.251443, -0.221384, -0.344603,
        0.283788,  0.284915,  0.057016,  0.384028, -0.733822,  0.303532,
       -0.583847, -0.02494 , -0.218198,  0.018208,  0.145667,  0.255729,
        0.498478, -0.112536, -0.047852, -0.337887,  0.031235, -0.432209,
        0.208977,  0.012221,  0.19098 , -0.206233, -0.113016,  0.111511,
       -0.059505,  0.027593, -0.100111, -0.230436, -0.078137,  0.278976,
        0.19794 ,  0.210459, -0.155359, -0.112397,  0.26439 , -0.199999,
        0.058368,  0.032107,  0.110054,  0.051876, -0.240713,  0.277577,
        0.289985, -0.200094, -0.101406,  0.131032, -0.059501,  0.134268,
        0.148928, -0.085373,  0.030669,  0.068627, -0.215063, -0.081981,
       -0.052877, -0.291941, -0.458875,  0.050618,  0.214273,  0.184849,
       -0.038208,  0.036236, -0.642125, -0.464941,  0.372085, -0.187011,
       -0.069729,  0.035051,  0.099067,  0.057524,  0.006579,  0.533439,
       -0.198236, -0.344194, -0.067796, -0.063289, -0.069952, -0.041706,
        0.142563,  0.05342 , -0.054871,  0.2577  ,  0.50673 ,  0.208496,
        0.650207, -0.252643,  0.320542, -0.54734 , -0.637634, -0.51038 ,
        0.025401,  0.05803 ,  0.16656 ,  0.257388,  0.305958, -0.219199,
        0.2086  , -0.251887, -0.185647, -0.027838, -0.186104, -0.070295,
       -0.094713,  0.102284, -0.123802,  0.083343,  0.315851, -0.031123,
        0.238093,  0.149717, -0.004316, -0.181932, -0.086956,  0.531083,
        0.112422,  0.231151,  0.242227,  0.012977, -0.095725, -0.283862,
       -0.061226,  0.311026,  0.146489,  0.054716, -0.048658, -0.282848,
       -0.054836, -0.030167, -0.090273, -0.435833, -0.287074, -0.351567,
       -0.002671, -0.020709, -0.276286,  0.281762, -0.114932, -0.243566,
       -0.188988,  0.007132,  0.146958, -0.269132, -0.034678,  0.164219,
       -0.090405,  0.044055, -0.405617,  0.162068,  0.008641,  0.142725,
       -0.065804,  0.136329,  0.247081, -0.144136, -0.007592,  0.113807,
        0.14899 , -0.012305, -0.346247,  0.111213,  0.011855,  0.250259,
        0.068319,  0.100238,  0.010446,  0.127471, -0.033004, -0.079141,
       -0.531317,  0.318433,  0.562701,  0.354442,  0.001789,  0.171425,
        0.010197,  0.043681, -0.031715,  0.505568, -0.453908,  0.050882,
       -0.294158, -0.279005, -0.121981,  0.182517,  0.419662,  0.325643,
       -0.027132, -0.020383,  0.133923, -0.047128,  0.406531, -0.385169,
       -0.148565, -0.109004, -0.062878, -0.236207,  0.008543,  0.044215,
        0.216761,  0.017197, -0.034889,  0.311272,  0.130243, -0.122846],
      dtype=float32)
```

Znajdziemy najbardziej podobne słowo do słowa kobieta.

```
wv.most_similar('kobieta')
```

```
[('mężczyzna', 0.7043285369873047),
 ('dziewczyna', 0.6813511252403259),
 ('ciężarna', 0.612939715385437),
 ('dziewczynka', 0.6094770431518555),
 ('50-letnia', 0.5805984139442444),
 ('młoda', 0.5788260102272034),
 ('40-letnia', 0.5631040930747986),
 ('kobietą', 0.5611501336097717),
```

```
 ('dama', 0.5526077747344971),
 ('mężczyzną', 0.5519220232963562)]
```

Znajdź najbardziej podobne słowo do słowa "kobieta", które nie jest podobne do słowa "mężczyzna"

```
wv.most_similar(positive=['kobieta'], negative=['mężczyzna'])
```

```
[('Teresa', 0.3597542345523834),
 ('Irlandka', 0.35389503836631775),
 ('Barbara', 0.33941537141799927),
 ('Helena', 0.33837634325027466),
 ('Eleonora', 0.32829549908638),
 ('mianowana', 0.3252837657928467),
 ('Maryse', 0.32269802689552307),
 ('Emmanuella', 0.32241636514663696),
 ('czarnoskóra', 0.3224010169506073),
 ('Izabela', 0.3222866654396057)]
```

W grupie "pies kot żubr budyń" znajdź słowo, które nie pasuje.

```
wv.doesnt_match(['pies', 'kot', 'żubr', 'budyń'])
```

```
'budyń'
```

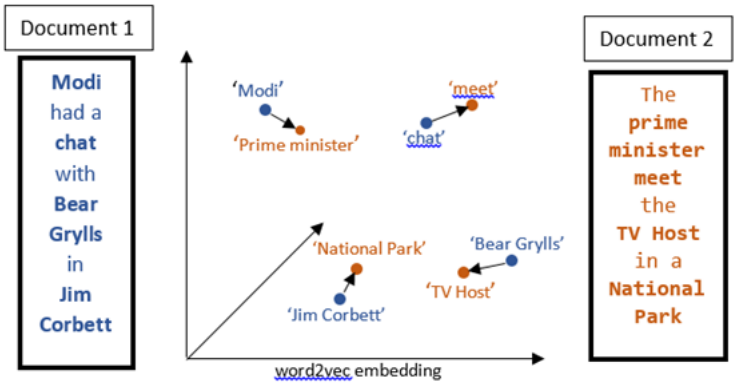
zbadaj podobieństwo między kotem a psem

```
wv.similarity('kot', 'pies')
```

```
np.float32(0.6970217)
```

Teraz zajmiemy się podobieństwami pomiędzy zdaniami

Word mover distance to minimalna suma odległości, jaką trzeba „pokonać", aby słowa z jednego zdania pokryły znaczeniowo słowa z drugiego zdania.




```
!pip install POT #Po tej instalacji gensim może szybciej i dokładniej liczyć wmdistanc
```

```
Collecting POT
  Downloading pot-0.9.7.post1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.2/44.2 kB 1.7 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.16 in /usr/local/lib/python3.13/dist-packages (from POT) (2.1.3)
Requirement already satisfied: scipy>=1.6 in /usr/local/lib/python3.13/dist-packages (from POT) (1.16.3)
Downloading pot-0.9.7.post1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (32.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 32.9/32.9 MB 10.3 MB/s eta 0:00:00
Installing collected packages: POT
Successfully installed POT-0.9.7.post1
```

```
sentence_1 = 'Byłem dzisiaj w kinie'.lower().split()
sentence_2 = 'Poszłam wczoraj do teatru'.lower().split()
```

```
wv.wmdistance(sentence_1, sentence_2)
```

```
1.176677407878612
```

WMD nie jest ograniczone do 0-1, ale przy normalizacji embeddingów zwykle wartości mieszczą się w 0-4 dla krótkich zdań.

wynik ~1.25 sugeruje, że zdania są umiarkowanie podobne semantycznie.

Dla porównania:

- Dwa identyczne zdania → WMD ≈ 0
- Zupełnie różne zdania → WMD > 2.5-3

Flair

Flair to biblioteka NLP od Zalando Research (Python)

Główna idea:

- Klasyczne embeddingi (Word2Vec, GloVe, FastText) dają stały wektor dla każdego słowa, niezależnie od kontekstu:

  "bank" w *"river bank"* i *"money bank"* → ten sam wektor.

- Flair tworzy kontekstowe embeddingi(wektor słowa zależy od jego zdania / kontekstu)

  więc "*bank*" w "*river bank*" będzie inny niż w "*money bank*".

Flair uczy się tego przez modele LSTM działające w dwóch kierunkach (forward i backward).

```
!pip install flair
```

```
Requirement already satisfied: torch>=1.13.1 in /usr/local/lib/python3.13/dist-packages (from flair) (2.11.0+cpu)
Requirement already satisfied: tqdm>=4.63.0 in /usr/local/lib/python3.13/dist-packages (from flair) (4.67.3)
Requirement already satisfied: transformer-smaller-training-vocab>=0.2.3 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: transformers<5.0.0,>=4.25.0 in /usr/local/lib/python3.13/dist-packages (from transformers[s
Requirement already satisfied: wikipedia-api>=0.5.7 in /usr/local/lib/python3.13/dist-packages (from flair) (0.16.0)
Requirement already satisfied: bioc<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from flair) (2.1)
Requirement already satisfied: jsonlines>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from bioc<3.0.0,>=2.0.0->flair
Requirement already satisfied: intervaltree in /usr/local/lib/python3.13/dist-packages (from bioc<3.0.0,>=2.0.0->flair) (3
Requirement already satisfied: docopt in /usr/local/lib/python3.13/dist-packages (from bioc<3.0.0,>=2.0.0->flair) (0.6.2)
Requirement already satisfied: botocore<1.44.0,>=1.43.114 in /usr/local/lib/python3.13/dist-packages (from boto3>=1.20.27-
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /usr/local/lib/python3.13/dist-packages (from boto3>=1.20.27->fla
Requirement already satisfied: s3transfer<0.20.0,>=0.19.0 in /usr/local/lib/python3.13/dist-packages (from boto3>=1.20.27-
Requirement already satisfied: wrapt<3,>=1.16 in /usr/local/lib/python3.13/dist-packages (from deprecated>=1.2.13->flair)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.13/dist-packages (from ftfy>=6.1.0->flair) (0.9.1)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.13/dist-packages (from gdown>=4.4.0->flair) (4.13.
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-packages (from gdown>=4.4.0->flair) (4.0.7)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.13/dist-packages (from gdown>=4.4.0->flair) (2.32
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10.0->
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10.0->f
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10.0->flair
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub
Requirement already satisfied: six in /usr/local/lib/python3.13/dist-packages (from langdetect>=1.0.9->flair) (1.17.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (0.
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (2.1
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (11.3.
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from mpld3>=0.3->flair) (3.1.6)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.0.2->flair) (
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.0.2->flair)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.0.2->
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch>=1.13.1->flair) (80.10
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch>=1.13.1->flair) (1.14.
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.13.1->flair) (3.7
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.13/dist-packages (from transformers<5
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.13/dist-packages (from transformers<5.0.0,>=4.
Requirement already satisfied: sentencepiece!=0.1.92,>=0.1.91 in /usr/local/lib/python3.13/dist-packages (from transformer
Requirement already satisfied: protobuf in /usr/local/lib/python3.13/dist-packages (from transformers[sentencepiece]<5.0.0
Requirement already satisfied: click==8.5.0 in /usr/local/lib/python3.13/dist-packages (from wikipedia-api>=0.5.7->flair)
Requirement already satisfied: httpx==0.28.1 in /usr/local/lib/python3.13/dist-packages (from wikipedia-api>=0.5.7->flair)
Requirement already satisfied: tenacity==9.1.4 in /usr/local/lib/python3.13/dist-packages (from wikipedia-api>=0.5.7->flai
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx==0.28.1->wikipedia-api>=0.5.7-
Requirement already satisfied: certifi in /usr/local/lib/python3.13/dist-packages (from httpx==0.28.1->wikipedia-api>=0.5.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx==0.28.1->wikipedia-api
Requirement already satisfied: idna in /usr/local/lib/python3.13/dist-packages (from httpx==0.28.1->wikipedia-api>=0.5.7->
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx==0.28.1->wi
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in /usr/local/lib/python3.13/dist-packages (from botocore<1.44.0
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=1.2.0->scikit-lea
Requirement already satisfied: attrs>=19.2.0 in /usr/local/lib/python3.13/dist-packages (from jsonlines>=1.2.0->bioc<3.0.0
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch>=1
Requirement already satisfied: accelerate>=0.26.0 in /usr/local/lib/python3.13/dist-packages (from transformers[sentencepi
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.13/dist-packages (from beautifulsoup4->gdown>=4.4.0
Requirement already satisfied: sortedcontainers in /usr/local/lib/python3.13/dist-packages (from intervaltree->bioc<3.0.0,
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->mpld3>=0.3->flair)
```

```
from flair.embeddings import FlairEmbeddings, StackedEmbeddings
from flair.data import Sentence
```

```
embeddings = FlairEmbeddings('news-forward')

#tworzymy objekty Sentence
s1 = Sentence("How does it feel to be a scribe?")
s2 = Sentence("There is no such thing as simply good or bad")
sentences = [s1, s2]


# Obliczamy embeddingi(zanurzamy słowa)
embeddings.embed(sentences)

for sentence in sentences: #Zanurzone zdania nadal są obiektem Sentence
  print(sentence)
```

```
Sentence[9]: "How does it feel to be a scribe?"
Sentence[10]: "There is no such thing as simply good or bad"
```

```
#dostajemy embeddingi poszczególnych słów
for sentence in sentences:
  for token in sentence:
    print(token.embedding)
#embeddingi są tensorami
```

```
tensor([-0.0011, -0.0021,  0.0038,  ..., -0.0004,  0.0099,  0.0547])
tensor([-0.0016, -0.0260,  0.0388,  ...,  0.0031, -0.0070,  0.0025])
tensor([0.0174, 0.0181, 0.0148,  ..., 0.0051, 0.0051, 0.0130])
tensor([ 0.0001,  0.0128,  0.0476,  ..., -0.0084, -0.0023,  0.0106])
tensor([-2.1672e-03,  8.3753e-05,  2.5102e-02,  ..., -2.8347e-04,
        -1.2090e-02,  2.9513e-02])
tensor([-0.0032,  0.0020,  0.0280,  ...,  0.0004, -0.0017, -0.0106])
tensor([ 5.3283e-03,  3.8049e-05, -2.2042e-02,  ..., -4.1646e-04,
         1.9279e-02, -1.2351e-02])
tensor([-1.0594e-03,  4.8670e-05,  2.1462e-02,  ..., -1.1294e-05,
        -4.3308e-03,  3.1280e-03])
tensor([ 8.7317e-04,  3.7862e-06, -4.6351e-04,  ..., -1.4222e-04,
         5.4708e-03,  3.0888e-03])
tensor([-0.0012, -0.0002, -0.1499,  ...,  0.0024,  0.0017,  0.1648])
tensor([ 0.0078, -0.0047,  0.0564,  ..., -0.0012,  0.0325,  0.0035])
tensor([ 5.8074e-02, -7.9190e-05,  9.8895e-02,  ..., -2.7518e-03,
         8.8326e-04,  2.6957e-02])
tensor([-0.0004, -0.0036,  0.0781,  ..., -0.0004, -0.0344, -0.0005])
tensor([ 0.0014,  0.0033, -0.0118,  ..., -0.0005, -0.0164,  0.0293])
tensor([ 1.2536e-02, -1.0569e-03, -1.1876e-02,  ..., -2.9878e-05,
         4.0988e-03,  2.8744e-02])
tensor([-0.0053, -0.0011,  0.0058,  ..., -0.0018,  0.0212,  0.0082])
tensor([ 2.0060e-04,  9.6346e-04, -2.2168e-02,  ...,  6.8233e-05,
        -1.3982e-01,  3.5051e-03])
tensor([-9.0032e-05, -1.7683e-03,  1.0521e-02,  ..., -1.9229e-03,
        -1.2467e-05,  3.2030e-02])
tensor([-0.0006,  0.0004, -0.0019,  ..., -0.0005,  0.0061, -0.0073])
```

```
import numpy as np
```

```
for sentence in sentences:
  for token in sentence:
    print(np.array(token.embedding))
```

```
[-0.00108573 -0.00213397  0.00377833 ... -0.00035715  0.0099433
  0.05467612]
[-0.00163258 -0.02604993  0.03880594 ...  0.00308893 -0.00699765
  0.00248321]
[0.01740974 0.01807804 0.0147752  ... 0.00507008 0.00508688 0.01302868]
[ 0.00013419  0.01282798  0.04760626 ... -0.00837356 -0.00231261
  0.01057426]
[-2.1671702e-03  8.3752588e-05  2.5102206e-02 ... -2.8347093e-04
 -1.2090425e-02  2.9513163e-02]
[-0.00319913  0.00195263  0.02804686 ...  0.00040722 -0.00174229
 -0.01062463]
[ 5.32832043e-03  3.80494457e-05 -2.20420863e-02 ... -4.16457391e-04
  1.92785766e-02 -1.23509625e-02]
[-1.0594028e-03  4.8669703e-05  2.1461956e-02 ... -1.1294257e-05
 -4.3307757e-03  3.1279572e-03]
[ 8.7317190e-04  3.7861769e-06 -4.6351264e-04 ... -1.4222013e-04
```

```
  5.4707765e-03  3.0888412e-03]
[-0.00118427 -0.00018427 -0.14988413 ...  0.00244023  0.00166776
  0.16482687]
[ 0.00780449 -0.00474085  0.056434   ... -0.00120921  0.03250852
  0.00350805]
[ 5.8073573e-02 -7.9189987e-05  9.8894574e-02 ... -2.7517818e-03
  8.8326377e-04  2.6957348e-02]
[-0.0003961  -0.00361209  0.07808607 ... -0.00039112 -0.03444917
 -0.00053953]
[ 0.00144947  0.00326684 -0.01180778 ... -0.00050383 -0.01641377
  0.02928468]
[ 1.2535800e-02 -1.0568546e-03 -1.1875577e-02 ... -2.9878282e-05
  4.0988261e-03  2.8744373e-02]
[-0.00529048 -0.00108928  0.00580226 ... -0.00176485  0.02117513
  0.00820003]
[ 2.0059671e-04  9.6345815e-04 -2.2167904e-02 ...  6.8233297e-05
 -1.3981722e-01  3.5051273e-03]
[-9.00320447e-05 -1.76826888e-03  1.05207525e-02 ... -1.92288996e-03
 -1.24673452e-05  3.20298225e-02]
[-0.00058445  0.00037377 -0.00193683 ... -0.00046098  0.00614759
 -0.00726028]
/tmp/ipykernel_842/932037997.py:3: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so passing cc
  print(np.array(token.embedding))
```

Embeddingi dla zdań

```
from flair.embeddings import DocumentPoolEmbeddings
```

```
sentence_embeddings = DocumentPoolEmbeddings([FlairEmbeddings('news-forward')])
```

```
sentence_embeddings.embed(sentences)
```

```
for sentence in sentences:
  print(sentence.embedding)
```

```
tensor([ 0.0016,  0.0005,  0.0175,  ..., -0.0001,  0.0014,  0.0104])
tensor([ 0.0073, -0.0008,  0.0052,  ..., -0.0007, -0.0124,  0.0289])
```

Zadania

Zadanie1

Załaduj polski model kontekstowych embeddingów Flair:

```
embeddings = FlairEmbeddings('pl-forward')
```

```
s1 = Sentence("Zamek w Malborku jest bardzo stary.")
s2 = Sentence("Suwak w kurtce to zamek błyskawiczny")
s3 = Sentence("Sprawdź, czy zamek w drzwiach działa")
sentences = [s1, s2, s3]
```

Oblicz embeddingi dla wszystkich zdań:

```
embeddings.embed(sentences)
```

```
[Sentence[7]: "Zamek w Malborku jest bardzo stary.",
 Sentence[6]: "Suwak w kurtce to zamek błyskawiczny",
 Sentence[7]: "Sprawdź, czy zamek w drzwiach działa"]
```

Wypisz każde zdanie i dla słowa "zamek", wypisz pierwsze 5 wymiarów embeddingu:

```
for sentence in sentences:
  #TODO
  print(sentence, end=' | ')
  for token in sentence:
    if token.text.lower() == 'zamek':
      print(token.embedding[:5])
```

```
Sentence[7]: "Zamek w Malborku jest bardzo stary." | tensor([-5.4509e-03, -2.1590e-02,  4.5458e-05, -1.7073e-04, -4.3133e-04
Sentence[6]: "Suwak w kurtce to zamek błyskawiczny" | tensor([-0.0043, -0.0131,  0.0004, -0.0001, -0.0004])
Sentence[7]: "Sprawdź, czy zamek w drzwiach działa" | tensor([-4.3021e-03, -2.1082e-02,  7.7413e-04, -6.9931e-05, -3.6798e-0
```

Czy embeddingi są różne w zależności od znaczenia słowa?

Odpowiedź: Tak

Zadanie 2

Stwórz bota z bazą przykładownych pytań i odpowiedzi, zwracającego odpowiedź na zapytanie na podstawie podobieństwa (WMD albo flair document embeddings) pomiędzy zapytniem użytownika a pytaniami w swojej bazie

Przykład:

Zbiór pytań i odpowiedzi :

```
[
  {"q1" : "a1"},
  {"q2" : "a2"},
  {"q3" : "a3"},
  ...
  {"qn" : "an"},
]
```

Wektoryzujemy pytania:

```
  {"q1_vecotorized" : "a1"},
  {"q2_vecotorized" : "a2"},
  {"q3_vecotorized" : "a3"},
  ...
  {"qn_vecotorized" : "an"},
```

Dla pytania użytkownia pytania q_user wektoryzujemy je i liczmy podobieństwo z pytaniami w bazie

```
sim("q_user_vectorized","q1_vecotorized"), sim("q_user_vectorized","q2_vecotorized"), ..
```

Zwracamy odpowiedź do pytania z bazy najbardziej podobnego do pytania użytkownika

$$argmax(sim(q_{user}vectorized, q_ivecotorized))$$

Do liczenia podobnieństma można użyć np https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cosine.html ale wtedy robimy argmin bo

$$cosine\ distance = 1 - cosine\ similarity$$

```
qa = [
  {"q": "Jakie są 3 główne czynniki wpływające na fotosyntezę?", "a": "Główne czynniki
  {"q": "Czym różni się stopa procentowa stała od zmiennej?", "a": "Stała stopa gwarant
  {"q": "Do czego służy protokół DNS w sieciach komputerowych?", "a": "Protokół DNS (Do
  {"q": "Kim był Juliusz Cezar i z czego zasłynął?", "a": "Rzymski wodz i polityk, któr
  {"q": "Co oznacza pojęcie ucieczki w przestrzeń kosmiczną, czyli druga prędkość kosmi
  {"q": "Czym charakteryzuje się styl architektoniczny gotyk?", "a": "Gotyk wyróżnia si
```

```
  {"q": "Jaka jest główna różnica między kawą Arabica a Robusta?", "a": "Arabica ma łag
]
```

```
# # importy
# from scipy.spatial.distance import cosine
# sentence_embeddings = DocumentPoolEmbeddings([FlairEmbeddings('news-forward')])

# def generate_sentence_embeddings_for_base_questions():
#   for q in qa:
#     q['embedding'] = get_sentence_embedding(q['q'])

# def get_sentence_embedding(input_str: str):
#   sentence = Sentence(input_str)
#   embeddings.embed(sentence)
#   mean_vector = np.mean([t.embedding.cpu().detach().numpy() for t in sentence], axis
#   return mean_vector

# def bot(user_question: str):
#   user_question_embedding = get_sentence_embedding(user_question)
#   final_q = None
#   final_a = None
#   max_sim = -1
#   for item in qa:
#     base_question_embedding = item['embedding']
#     current_sim = calculate_similarity(user_question_embedding, base_question_embedd
#     if current_sim > max_sim:
#       max_sim = current_sim
#       final_q = item['q']
#       final_a = item['a']

#   print(f"Pytanie użytkownika:", user_question)
#   print(f"Najbardziej podobne pytanie z bazy danych:", final_q)
#   print(f"Odpowiedź:", final_a)

# def calculate_similarity(vec1, vec2):
#     return 1 - cosine(vec1, vec2)
```

```
from scipy.spatial.distance import cosine

sentence_embeddings = DocumentPoolEmbeddings([FlairEmbeddings('news-forward')])

def generate_sentence_embeddings_for_base_questions():
  for q in qa:
    sentence = Sentence(q['q'])
    sentence_embeddings.embed(sentence)
    q['embedding'] = sentence.get_embedding().cpu().detach().numpy()

def calculate_similarity(a, b):
  norm_a = np.linalg.norm(a)
  norm_b = np.linalg.norm(b)
  if norm_a == 0 or norm_b == 0:
    return 0
  return np.dot(a, b) / (norm_a * norm_b)

def bot(user_question: str):
  user_question_sentence = Sentence(user_question)
  sentence_embeddings.embed(user_question_sentence)
  user_question_embedding = user_question_sentence.get_embedding().cpu().detach().nump
  final_q = None
  final_a = None
  max_sim = -1
  for item in qa:
```

```
        base_question_embedding = item['embedding']
        current_sim = calculate_similarity(user_question_embedding, base_question_embeddin
        print(current_sim)
        if current_sim > max_sim:
          max_sim = current_sim
          final_q = item['q']
          final_a = item['a']

        print("Retrieving with cosine ...
```